

Advanced

Deep Learning Use Cases

Transfer Learning · Improving Model Accuracy · Real-World CV Applications

Bridging theory and production — for college students

PyTorch

Keras/TF

torchvision

OpenCV

ONNX

ADV
DL

What We'll Cover Today

01

Transfer Learning Intro

Why train from scratch when pretrained models exist?

02

Fine-Tuning Deep Models

Freeze, unfreeze, layer-by-layer strategy

03

Improving Model Accuracy

Augmentation, regularisation, LR scheduling

04

Hyperparameter Tuning

Manual search, grid search, Optuna/Ray Tune

05

Real-World CV — Detection

Object detection: YOLO, Faster R-CNN in practice

06

Real-World CV — Segmentation

Semantic & instance segmentation: U-Net, Mask R-CNN

07

Real-World CV — Industries

Medical imaging, autonomous driving, manufacturing

08

Deployment & Production

ONNX, TorchScript, TFLite, FastAPI serving

Transfer Learning — Overview

Reuse what a model already knows. Train only what it needs to learn.

Scenario: You want to classify 10 types of skin lesions. You have 2,000 images. Training a ResNet-50 from scratch would take days and overfit. Transfer learning lets you reach >90% accuracy in under an hour.

Why Transfer Learning Works

Early layers learn universal features

Edges, corners, curves — same in any image domain.

Middle layers learn shapes & textures

Wheels, fur, fabric — transferable across tasks.

Deep layers learn task-specific concepts

Only these need re-training for your new task.

Pretrained on 1M+ images

ImageNet-trained models encode rich visual priors that take years to re-learn.

10x-100x less data needed

A few hundred labelled examples can outperform training from scratch with thousands.

Feature Extraction

Freeze ALL pretrained layers.
Add new classifier head.
Only head is trained.

Best for: very small datasets,
similar domain to ImageNet.

Fine-Tuning

Freeze early layers.
Unfreeze later layers + head.
Train end-to-end with small LR.

Best for: medium datasets,
different domain.

Full Retraining

Unfreeze ALL layers.
Use pretrained weights
as initialisation.

Best for: large datasets,
very different domain.

Fine-Tuning a Pretrained Model

Step-by-step: Load → Freeze → Adapt → Train

Scenario: Fine-tune ResNet-50 pretrained on ImageNet to classify 5 types of plant disease from 3,000 leaf images.

PyTorch `fine_tune.py`

```
import torch, torchvision
import torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder

# Step 1: Load pretrained ResNet-50
model = models.resnet50(weights='IMAGENET1K_V2')

# Step 2: Freeze ALL layers
for param in model.parameters():
    param.requires_grad = False

# Step 3: Replace the classifier head (fc layer)
num_classes = 5 # plant disease classes
model.fc = nn.Sequential(
    nn.Dropout(0.4),
    nn.Linear(model.fc.in_features, 256),
    nn.ReLU(),
    nn.Linear(256, num_classes)
)

# Step 4: Unfreeze last residual block for fine-tuning
for param in model.layer4.parameters():
    param.requires_grad = True

# Step 5: Differential learning rates
```

Why freeze first?

Avoids 'catastrophic forgetting' — the pretrained weights encode millions of training hours. Destroying them immediately wastes everything.

Differential LR

Pretrained layers: 10x smaller LR (1e-4)
New head: normal LR (1e-3)
Preserves learned features, trains new ones.

Layer unfreezing

Progressively unfreeze: head first, then residual blocks from deep to shallow. Monitor val loss at each stage.

ImageNet normalise

Always use ImageNet mean/std:
mean=[0.485,0.456,0.406]
std=[0.229,0.224,0.225]

Input size

ResNet expects 224x224. Use `transforms.Resize(256)` then `transforms.CenterCrop(224)`.

Pretrained Model Zoo

Choose the right backbone for your task and constraints

Model	Year	Params	Top-1 Acc	Speed	Best Used For
ResNet-50	2015	25M	76.1%	Fast	General purpose baseline. Great starting point.
EfficientNet-B4	2019	19M	82.9%	Fast	Best accuracy/param trade-off. Mobile-friendly.
ViT-B/16	2020	86M	81.8%	Slow	Vision Transformer. Needs large datasets.
ConvNeXt-B	2022	89M	83.8%	Med	CNN that matches ViT. Excellent for fine-tuning.
CLIP ViT-L	2021	307M	—	Slow	Zero-shot via text prompts. Multimodal.
DINOv2-L	2023	307M	86.3%	Med	Self-supervised. No labels needed for fine-tuning.

Recommendation: Start with EfficientNet-B4 or ResNet-50 for most CV tasks. Use CLIP for zero-shot or multimodal needs. DINOv2 if you lack labels.

```
torchvision.models.resnet50(weights='IMAGENET1K_V2') | timm.create_model('efficientnet_b4', pretrained=True) |  
transformers.CLIPModel.from_pretrained('openai/clip-vit-large-patch14')
```

Improving Model Accuracy

Augmentation · Regularisation · Learning Rate Scheduling

Scenario: Your fine-tuned model hits 78% validation accuracy but training accuracy is 96%. Classic overfitting. Here are your levers.

accuracy_tricks.py

```
from torchvision import transforms
import torch.optim as optim

# Data Augmentation pipeline
train_tf = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.2),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.RandomRotation(degrees=15),
    transforms.RandomGrayscale(p=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
    transforms.RandomErasing(p=0.2), # Cutout regularisation
])

# MixUp augmentation (advanced)
def mixup_data(x, y, alpha=0.4):
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0))
    return lam*x + (1-lam)*x[idx], y, y[idx], lam

# Learning rate scheduling
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=50, eta_min=1e-6)

# Label smoothing (prevents overconfidence)
```

Random Erasing

Cuts out random patches.
Forces model to use context
not just one strong region.

MixUp

Blends two images + labels.
Improves calibration &
robustness to noisy labels.

CosineAnneal LR

LR decreases as cos curve.
Avoids harsh drops, helps
escaping sharp minima.

Label Smoothing

Soft targets: 0.9 not 1.0.
Prevents overconfident
predictions, better generalis.

Weight Decay

L2 penalty on weights.
Add via optimizer:
weight_decay=1e-4

Hyperparameter Tuning

From manual guessing to automated search — find your best model faster

Scenario: You need to tune LR, batch size, dropout, and weight decay for your plant classifier. Manual search takes weeks. Optuna does it overnight.

optuna_tuning.py

```
import optuna
import torch.nn as nn

def objective(trial):
    # Sample hyperparameters
    lr = trial.suggest_float('lr', 1e-5, 1e-2, log=True)
    batch_size = trial.suggest_categorical('batch_size', [16, 32, 64])
    dropout = trial.suggest_float('dropout', 0.1, 0.5)
    weight_decay = trial.suggest_float('wd', 1e-5, 1e-2, log=True)
    optimizer_name = trial.suggest_categorical('optimizer', ['Adam', 'SGD'])

    # Build model with sampled params
    model = build_model(dropout=dropout)
    optimizer = getattr(torch.optim, optimizer_name)(
        model.parameters(), lr=lr, weight_decay=weight_decay)

    # Train and return val accuracy
    val_acc = train_and_evaluate(model, optimizer, batch_size)
    return val_acc

# Run 100 trials with pruning (stops bad trials early)
study = optuna.create_study(direction='maximize',
    pruner=optuna.pruners.MedianPruner())
study.optimize(objective, n_trials=100)

print('Best params:', study.best_params)
print('Best val acc:', study.best_value)
```

Manual / Grid Search

Try a few combinations by hand or exhaustively.
Feasible for 1-2 hyperparams.
Exponential cost with more params.

Random Search

Sample param combos randomly.
2-3x more efficient than grid.
Good baseline — always try this first.

Bayesian Optimisation

Builds a probabilistic model of the objective.
Optuna, Ax, BayesOpt.
Best sample efficiency for expensive evals.

Population-Based

Evolves a population of configs.
Ray Tune PBT — great for RL + large models.
Most complex but handles non-stationarity.

Real-World CV: Object Detection

Not just 'what' is in the image — but 'where'

Scenario: Build a retail shelf monitor that detects and counts every product on a supermarket shelf in real-time from CCTV footage.

Model	Type	Latency	COCO mAP	Best For
YOLOv8	One-stage	1-5ms	50.2 mAP	Real-time apps, edge devices, drones
Faster R-CNN	Two-stage	50-200ms	47.0 mAP	Accuracy-critical, medical imaging
DETR	Transformer	50ms	42.0 mAP	End-to-end, no anchor tuning needed
RT-DETR	Hybrid	10ms	53.1 mAP	Real-time + transformer accuracy

```
yolo_detection.py

# YOLOv8 inference - retail shelf detection
from ultralytics import YOLO

model = YOLO('yolov8m.pt')          # medium model

# Fine-tune on custom shelf dataset
model.train(
    data='shelf_dataset.yaml',      # classes: product_A, product_B ...
    epochs=100,
    imgsz=640,
    batch=16,
    lr0=0.01,
```


Real-World CV: Segmentation

Pixel-level understanding — semantic, instance, and panoptic

Scenario: Segment tumour regions in MRI scans pixel-by-pixel to assist radiologists in diagnosis. Every misclassified pixel matters.

Semantic Segmentation

Assigns a class label to every pixel.
All 'cat' pixels = same label.
Ignores individual instances.

FCN, DeepLab, SegFormer

Instance Segmentation

Detects AND segments each object individually. Two cats = two masks.
Outputs per-object masks.

Mask R-CNN, YOLO-seg

Panoptic Segmentation

Combines both: semantic labels for background (sky, road) + instance masks for objects.

Panoptic FPN, Mask2Former

unet_segmentation.py

```
import segmentation_models_pytorch as smp

# U-Net with EfficientNet encoder for MRI segmentation
model = smp.Unet(
    encoder_name      = 'efficientnet-b4',
    encoder_weights   = 'imagenet',      # pretrained!
    in_channels       = 1,               # grayscale MRI
    classes           = 3,               # background, tissue, tumour
    activation        = None             # raw logits
)
```

```
# Dice + BCE combined loss (standard for medical segmentation)
```

Real-World CV Applications by Industry

Where deep learning vision is changing the world right now

Healthcare

- > Chest X-ray: pneumonia / COVID-19 detection (CheXNet, 94%+ AUC)
- > Retinal scan: diabetic retinopathy grading
- > Pathology slides: cancer cell detection at scale
- > MRI/CT: tumour segmentation (U-Net variants)
- > Colonoscopy: polyp detection in real-time

Autonomous Driving

- > Lane detection from dashcam (OpenCV + CNN)
- > Pedestrian & vehicle detection (YOLO, Faster R-CNN)
- > Road sign classification (fine-tuned ResNet)
- > Depth estimation from stereo cameras
- > HD map building with LiDAR + camera fusion

Manufacturing

- > Surface defect detection on production lines
- > PCB inspection: solder joint quality
- > Pill counting and packaging verification
- > Weld quality assessment from thermal images
- > Robot arm guidance via visual servo

Retail & E-commerce

- > Product recognition from photos (visual search)
- > Shelf occupancy monitoring (YOLO)
- > Cashierless checkout (Amazon Go-style)
- > Customer flow analysis with heatmaps
- > Counterfeit detection in luxury goods

Deploying CV Models to Production

From Jupyter Notebook to real-world API in 5 steps

Scenario: Your plant disease classifier is trained. Now package it as a REST API so a mobile app can send leaf photos and get predictions.

1

Export

`torch.onnx.export()`
or `TorchScript`
`save_model()`

2

Optimise

Quantize INT8
Prune weights
ONNX Runtime

3

Package

FastAPI endpoint
Dockerize
CI/CD pipeline

4

Deploy

AWS/GCP/Azure
Edge device
Raspberry Pi

5

Monitor

Drift detection
A/B testing
Latency alerts

deploy_api.py

Step 1 - Export to ONNX

```
torch.onnx.export(model, dummy_input, 'plant_clf.onnx',  
                  input_names=['image'], output_names=['logits'],  
                  dynamic_axes={'image': {0: 'batch'}})
```

Step 2 - FastAPI serving endpoint

```
from fastapi import FastAPI, File, UploadFile  
import onnxruntime as ort, numpy as np  
from PIL import Image; import io
```

```
app = FastAPI()  
session = ort.InferenceSession('plant_clf.onnx')
```

```
@app.post('/predict')  
async def predict(file: UploadFile = File(...)):  
    img = Image.open(io.BytesIO(await file.read())).convert('RGB')  
    img = onnxruntime_image(img) # resize + normalize
```

ONNX Benefits

Framework-agnostic export.
Run on any runtime: TFLite,
ONNX Runtime, TensorRT

TorchScript

Serialize PyTorch models
without Python interpreter.
Faster inference, mobile ready

Quantization

INT8 quantization: 4x
smaller model, 2-4x faster.
Accuracy drop usually <1%

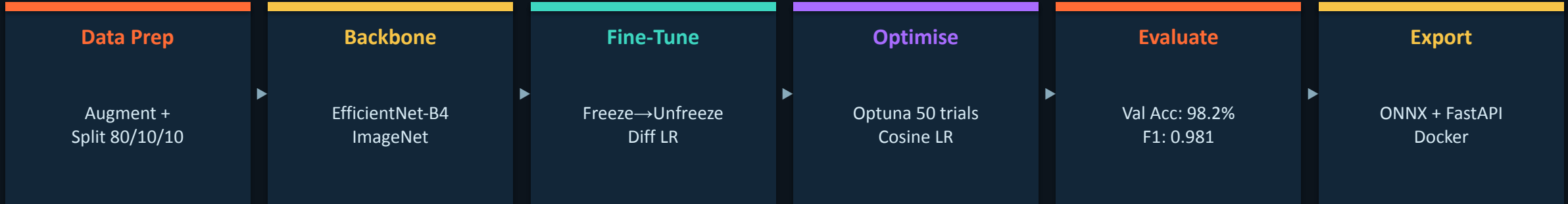
Docker + K8s

Containerise your FastAPI app.
Scale horizontally with
Kubernetes for high traffic.

Mini Project — Full Pipeline

Plant Disease Classifier: Transfer Learning → Accuracy Boost → Deployment

Build and deploy a plant disease classifier using transfer learning on PlantVillage dataset (54K images, 38 classes) — from raw images to a live REST API.



plant_disease_pipeline.py

```
# Complete pipeline (condensed)
import timm, torch, torch.nn as nn, optuna
from torchvision import transforms, datasets

# 1. Data
train_ds = datasets.ImageFolder('data/train', transform=aug_transforms)
val_ds   = datasets.ImageFolder('data/val',   transform=val_transforms)

# 2. Model: EfficientNet-B4 + custom head
backbone = timm.create_model('efficientnet_b4', pretrained=True, num_classes=0)
model     = nn.Sequential(backbone, nn.Dropout(0.3), nn.Linear(1792, 38))

# 3. Phase 1 - Head only (5 epochs)
for p in backbone.parameters(): p.requires_grad = False
train(model, epochs=5, lr=1e-3)

# 4. Phase 2 - Full fine-tune (25 epochs)
for p in model.parameters(): p.requires_grad = True
```

98.2

%
Val Accuracy
(38 classes)

<15m

\$
CPU
Inference
per image

54K

Training
Images

3h

Total Training
Time (GPU)

Next Steps

- > Vision Transformers: ViT, Swin, DINOv2 — attention-based vision
- > Semi-Supervised: Train on unlabelled data via pseudo-labels
- > 3D Vision: Point clouds, depth maps, NeRF rendering
- > Video Understanding: Action recognition, optical flow
- > Foundation Models: CLIP, SAM, Florence for zero-shot tasks

Summary & Key Takeaways

From pretrained backbones to production — the full advanced DL workflow

Key Takeaways

- + Transfer learning: reuse ImageNet weights to reach >95% accuracy with <5K images
- + Fine-tuning strategy: freeze layers → replace head → differential LR → progressive unfreeze
- + Accuracy levers: augmentation, label smoothing, cosine LR, MixUp, weight decay
- + Hyperparameter tuning: Optuna Bayesian search beats manual grid search every time
- + Object detection: YOLOv8 for speed (<5ms), Faster R-CNN for accuracy (medical)
- + Segmentation: U-Net + pretrained encoder for pixel-level tasks (Dice + BCE loss)
- + Production: ONNX export → FastAPI → Docker → monitor for data drift
- + Industry impact: Healthcare, autonomous driving, manufacturing, retail — all active now

Advanced Next Steps

Vision Transformers

Study Swin-T & DINOv2 architectures

Semi-Supervised DL

Pseudo-labels, Mean Teacher, FixMatch

Multimodal Models

CLIP, BLIP-2, LLaVA — vision + language

3D Computer Vision

NeRF, Gaussian splatting, point clouds

MLOps for CV

MLflow, Weights & Biases, DVC pipelines